

Observability — Logging, Metrics, Tracing

Phase 3 · Advanced · Estimated time: 1.5 hours

1. Concept From First Principles

Every pattern in this program — sharding, microservices, sagas, multi-region deployment — makes a system harder to understand from the outside once something goes wrong. In a monolith on one server, you can often just read a stack trace. In a system with dozens of sharded, replicated, asynchronously-communicating microservices spread across regions, a single user-facing error might originate three services and two message queues away from where it was observed. **Observability** is the discipline and tooling for understanding a system's internal state purely from its external outputs — and this chapter covers its three foundational pillars.

Logs: what happened, in detail

Logs are discrete, timestamped records of specific events ("user 4521 login failed: invalid password," "payment charge succeeded for order 8821"). They're the most granular and detailed observability signal, but also the most expensive to store and search at volume, and the hardest to usefully query across many services unless they're **structured** (consistent, machine-parseable key-value fields, like JSON, rather than free-form text) and **centralized** (aggregated into one searchable system — the ELK stack, i.e. Elasticsearch/Logstash/Kibana, being a common choice, directly reusing Phase 2 Chapter 13's inverted-index search infrastructure for log search specifically).

Metrics: how much, over time

Metrics are numeric measurements aggregated over time (requests per second, p99 latency, error rate, CPU utilization) — far cheaper to store than raw logs because they're pre-aggregated numbers rather than full event records, and well-suited to dashboards, trend analysis, and automated **alerting** (notify on-call when error rate exceeds 5% for more than 2 minutes). Metrics directly reuse Phase 1 Chapter 13's SLI/SLO vocabulary: your SLIs (the measured indicators, like p99 latency) are quite literally metrics, and Chapter 5 of this Phase's time-series databases are the standard storage engine underneath a metrics system, since "a numeric value at a point in time, queried by time range" is exactly what time-series databases are built for.

Traces: where, across services

Distributed tracing answers the question logs and metrics alone can't: for one specific user request that touched five microservices, where exactly did the time go, and where did it fail? Each request is tagged with a unique **trace ID** (generated at the entry point — typically the API gateway from Phase 2 Chapter 12) that gets propagated through every downstream service call. Each individual unit of work within that trace (a database query, an internal service call) is recorded as a **span**, with its own start time, duration, and metadata, and spans are nested to reconstruct the full call tree. The result is a visual, chronological map of exactly which service, in a request touching many services, was slow or failed — something that would otherwise require manually cross-referencing timestamps across five separate services' individual log files.

Backend engineer analogy

If you've ever added a **request_id** or correlation ID to every log line in a Laravel app (via middleware that generates one per request and includes it in every subsequent log entry), so that grepping one ID pulls every log line related to that single request, you've already built a primitive version of distributed tracing's trace ID concept — just without the automatic span timing, nesting, and cross-service propagation a real tracing system (Jaeger, Zipkin, OpenTelemetry) provides out of the box.

The practical, widely-quoted rule of thumb for using all three together: use **metrics** to know *something* is wrong (an alert fires because error rate spiked), use **traces** to see *where* it went wrong (which specific service in the request chain is failing or slow), and use **logs** to understand *why* it went wrong (the specific error message, stack trace, or business logic detail from that specific service). Mature observability platforms (Datadog, Grafana with Loki/Tempo/Prometheus, or a self-hosted OpenTelemetry stack) let you jump directly between these three signals — from a metric spike, to the trace that caused it, to the exact log line explaining it — rather than treating them as three disconnected tools.

A cost and volume consideration worth naming explicitly, since it comes up in real production systems constantly: capturing a full trace and complete structured logs for *every single request* at high traffic volume is expensive to store and process. Real systems use **sampling** — tracing only a percentage of requests (often more heavily sampling slow or errored requests specifically, since those are the most valuable to have full traces for) rather than 100% of traffic, trading some observability completeness for a manageable cost, similar in spirit to this Phase's Chapter 10 trading exactness for efficiency.

2. Real-World Example / Case Study

Razorpay's and PhonePe's payment-processing pipelines, touching multiple internal services (validation, fraud check, ledger, notification) per transaction, depend on distributed tracing to diagnose which specific service is adding latency during an incident — without a trace ID connecting all five services' logs for one transaction, an on-call engineer would need to manually correlate timestamps across five separate log streams during a live incident, which is far too slow.

Swiggy's and Zomato's platform teams rely on metrics-driven alerting (order-placement error rate, delivery-partner assignment latency) to catch problems within minutes during peak lunch and dinner rush, well before enough customers complain for the issue to surface through support tickets — the difference between a metric-driven alert firing in 2 minutes versus a support-ticket-driven discovery an hour later can be the difference between a minor blip and a genuinely damaging outage.

A Careem or Talabat engineering team investigating a spike in failed ride/order requests would typically start from a metrics dashboard (which service's error rate spiked, and when), jump to a handful of traces from that time window to see exactly which downstream dependency was failing, then pull the specific logs from that one service to read the actual error detail — the three-pillar workflow this chapter describes, applied directly.

3. Diagram

```
Distributed trace for one user request (trace_id: abc123):

[API Gateway] span: 240ms total
|
+--[Orders Service] span: 30ms
|
+--[Payments Service] span: 180ms <-- the slow one
|
|   +--[Fraud Check] span: 150ms <-- root cause found here
|   +--[Ledger Write] span: 20ms
|
+--[Notification Svc] span: 15ms

Metric alert fired: "p99 latency > 500ms for payments API" (WHAT)
Trace shows: Fraud Check span is the bottleneck (WHERE)
Log from Fraud Check service: "external fraud API timeout" (WHY)
```

4. Trade-offs Table

Signal	Answers	Storage cost	Best for
Logs	What exactly happened	High — full detail per event	Deep-dive debugging of a specific failure
Metrics	How much, over time	Low — pre-aggregated numbers	Dashboards, trend detection, alerting
Traces	Where, across services	Moderate to high — often sampled	Pinpointing which service caused latency/errors

5. Common Interview Questions

Q1: "Explain the difference between logs, metrics, and traces, and how you'd use them together during an incident."

Model approach: Use the 'metrics tell you something's wrong, traces tell you where, logs tell you why' framing, and walk through a concrete incident-response sequence using all three in order.

Q2: "How does distributed tracing work across multiple microservices?"

Model approach: Describe the trace ID generated at the entry point and propagated through every downstream call, with each unit of work recorded as a nested span, reconstructing the full call tree and timing breakdown for one specific request.

Q3: "Why not just capture full traces and complete logs for 100% of requests?"

Model approach: Name the storage and processing cost directly at high request volume, and introduce sampling (often biased toward slow/errorred requests) as the standard mitigation, trading some completeness for manageable cost.

Q4: "What storage technology underlies a metrics system, and why?"

Model approach: Name time-series databases (Chapter 5) directly, and explain that metrics are exactly the 'numeric value at a point in time, queried by time range' shape those databases are purpose-built for.

6. Practice Exercises

- 1 Design a trace-ID propagation scheme for a request that flows through an API gateway, an orders service, and a payments service, specifying how the ID is passed at each hop.
- 2 Given a scenario where p99 latency alerts fire for a checkout API, describe your step-by-step diagnostic process using metrics, traces, and logs in order.
- 3 Explain why sampling only 1% of all requests, but 100% of errored/slow requests, is a reasonable tracing strategy for a high-traffic system.

7. Curated YouTube Videos

Language	Video & Channel	Why it helps
English	"Monitoring & Observability Explained: Metrics, Logging & Tracing for Beginners"	Clear introduction to all three pillars covered in this chapter.
English	"Observability: Introduction, architecture and Logs, Metrics & Traces Explained (Ep. 01)"	Deeper architectural look at how the three pillars fit together.
Hindi	"System Design Complete Course 2026" (Hindi + English, 10+ hrs)	Covers observability as part of a full bilingual system design course.
Hindi	"Complete System Design Roadmap 2026 HLD & LLD" — CodeHelp	Places logging/metrics/tracing in context of the broader Hindi HLD roadmap.

8. Key Takeaways

- Observability is understanding a system's internal state from its external outputs: logs, metrics, and traces.
- Logs give detailed, per-event records but are expensive to store and hard to correlate across services unstructured.
- Metrics are cheap, pre-aggregated numbers over time, ideal for dashboards, trend analysis, and alerting.
- Distributed tracing uses a trace ID propagated across services, with nested spans reconstructing the full request path.
- Rule of thumb: metrics tell you something's wrong, traces tell you where, logs tell you why.
- Time-series databases (Chapter 5) are the natural storage engine underneath a metrics system.
- Sampling (especially biased toward slow/errored requests) keeps tracing and logging costs manageable at scale.